

Sequence to Sequence Networks and Attention

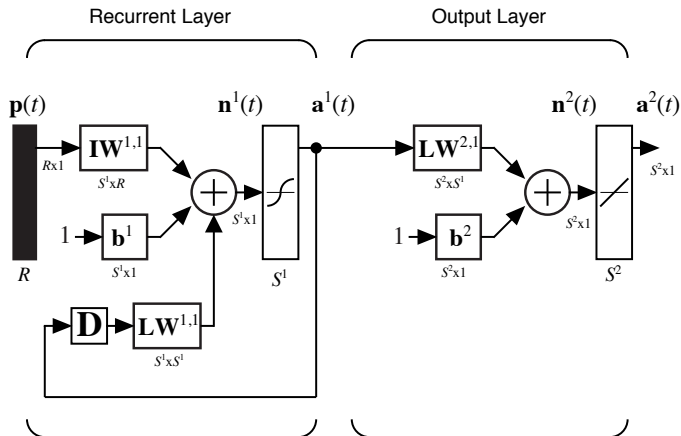
Deep Learning



Sequence to sequence (Seq2Seq) networks

- Seq2Seq is a family of networks.
- A sequence is input to the network, which then produces an output sequence.
- Originally developed for machine translation – input sequence is a sentence in language A and the output sequence is the sentence in language B.
- The network has two parts:
 - Encoder processes the input sequence and produces context.
 - Decoder generates the output sequence from the context.
- Many encoder and decoder structures have been used.
 - Originally LSTM networks used for encoder & decoder
 - Then attention added between encoder & decoder
 - Recurrent connections removed – attention-only transformer





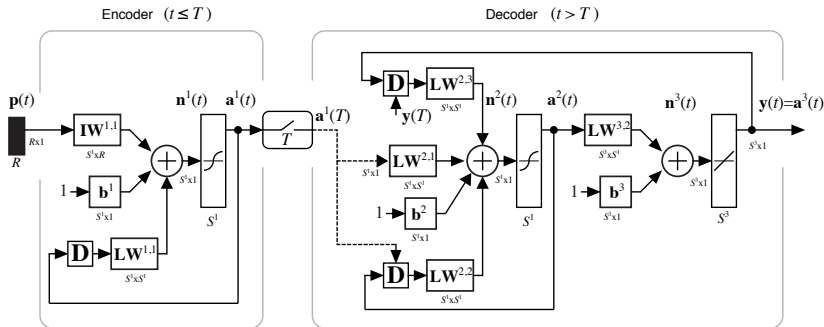
We will start by using RNNs for encoder and decoder.

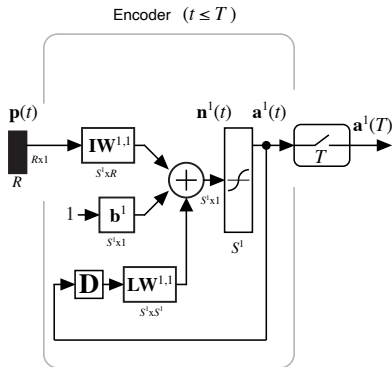
$$\mathbf{a}^1(t) = \text{tansig}(\mathbf{IW}^{1,1} \mathbf{p}(t) + \mathbf{LW}^{1,1} \mathbf{a}^1(t-1) + \mathbf{b}^1)$$

$$\mathbf{a}^2(t) = \mathbf{LW}^{2,1} \mathbf{a}^1(t) + \mathbf{b}^2$$



Sequence-to-sequence (Seq2Seq) network

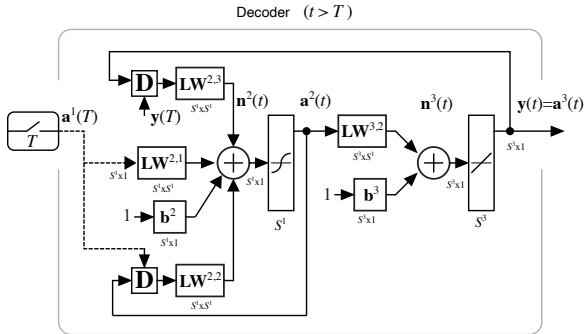




- The encoder takes the input sequence, which has T time points (words, or word fragments), and produces a layer output $a^1(t)$.
- This output is sampled at the final time point $t = T$.
- The output of the sampler is fixed and does not change with time.

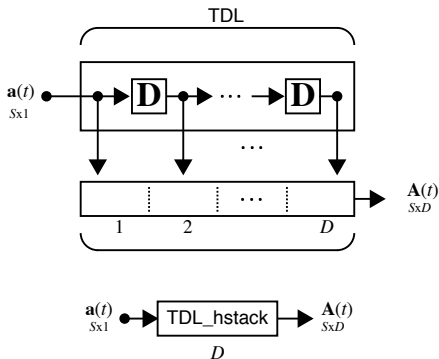


Seq2Seq decoder



- The final encoder state initializes the decoder state.
- It is also a constant input to the first layer of the decoder.
- This final encoder state is referred to as the context.
- The context summarizes the input sequence.





$$\mathbf{A}^1(t) = [\mathbf{a}^1(t) \quad \mathbf{a}^1(t-1) \quad \dots \quad \mathbf{a}^1(t-D+1)]$$

- The encoder recurrent layer has infinite memory.
- It may not store information most efficiently.
- For more flexibility, add a TDL of previous encoder states.



- Combine the $\mathbf{A}^1(t)$ vectors to form a new context.
- How much should each previous encoder state contribute?
- We want the context to change at each time step.
- When translating an English sentence to French, each French word may depend on different combinations of English words.
- How correlated is the previous decoder state to each previous encoder state.
- Find the inner product between the previous decoder state and all previous encoder states.

$$\mathbf{n}^4(t) = [\mathbf{A}^1(T)]^T \mathbf{a}^2(t-1)$$

- Normalize with the `softmax`

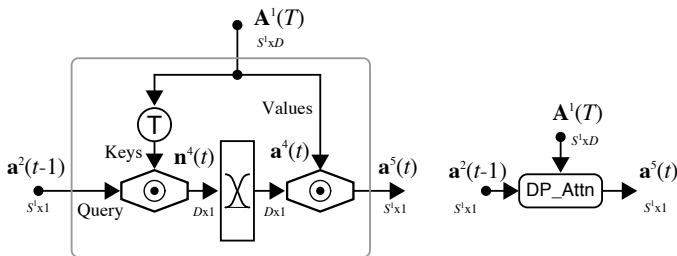
$$\mathbf{a}^4(t) = \text{softmax}(\mathbf{n}^4(t))$$

- Combine according to the amount of correlation.

$$\mathbf{a}^5(t) = \mathbf{A}^1(T) \mathbf{a}^4(t)$$



Dot product attention



- Attention consists of operations between a **query** vector and sets of **key** and **value** pairs.
- The query is compared to each of the keys to determine how much of each corresponding value to include in the result.
- In our case, the query is the previous decoder state $\mathbf{a}^2(t-1)$, the keys are all previous encoder states, and the values are also the previous encoder states.



Database meaning of query, keys and values

- Query: Floy

Rank	Last Name (Key)	Age (Value)
2nd	Flowers	35
1st	Floyd	24
3rd	Fly	57
...	...	...

- Compare the Query to all of the Keys in the database.
- Return the Value associated with the closest matching Key.



Example of attention

$$\mathbf{A}^1(T) = \begin{bmatrix} 0.7 & -0.6 & 0.2 & -0.8 \\ 0.3 & 0.5 & -0.7 & -0.5 \\ -0.6 & 0.1 & 0.6 & 0.4 \end{bmatrix} \quad \text{Keys/Values}$$

$$\mathbf{a}^2(t-1) = \begin{bmatrix} 0.4 \\ 0.1 \\ -0.3 \end{bmatrix} \quad \text{Query}$$

$$\mathbf{n}^4(t) = [\mathbf{A}^1(T)]^T \mathbf{a}^2(t-1) = \begin{bmatrix} 0.49 \\ -0.22 \\ -0.17 \\ -0.49 \end{bmatrix}$$

$$\mathbf{a}^4(t) = \text{softmax}(\mathbf{n}^4(t)) = \begin{bmatrix} 0.420 \\ 0.206 \\ 0.217 \\ 0.157 \end{bmatrix} \quad \text{Attention Weights}$$

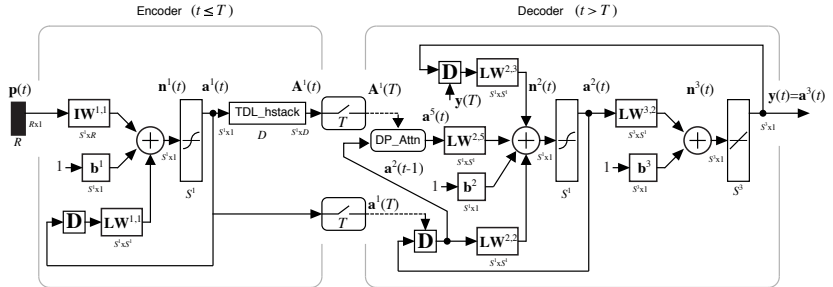


Using attention weights to combine values

$$\begin{aligned}\mathbf{a}^5(t) &= \mathbf{A}^1(T)\mathbf{a}^4(t) \\ &= 0.420 \begin{bmatrix} 0.7 \\ 0.3 \\ -0.6 \end{bmatrix} + 0.206 \begin{bmatrix} -0.6 \\ 0.5 \\ 0.1 \end{bmatrix} + 0.217 \begin{bmatrix} 0.2 \\ -0.7 \\ 0.6 \end{bmatrix} + 0.157 \begin{bmatrix} -0.8 \\ -0.5 \\ 0.4 \end{bmatrix} \\ &= \begin{bmatrix} 0.087 \\ -0.002 \\ -0.038 \end{bmatrix}\end{aligned}$$



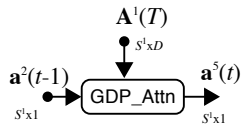
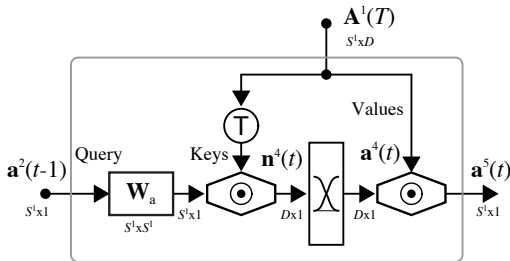
Seq2Seq network with attention



General dot product attention

- For vectors, $\mathbf{x}$ and $\mathbf{y}$, the standard inner product is $\mathbf{x}^T \mathbf{y}$.
- Another common inner product is the weighted dot product $\mathbf{x}^T \mathbf{W} \mathbf{y}$
- The symmetric, positive definite weighting matrix $\mathbf{W}$ is used to emphasize certain components of the dot product.
- This concept can be easily incorporated into the attention mechanism.

$$\mathbf{n}^4(t) = [\mathbf{A}^1(T)]^T \mathbf{W}_a \mathbf{a}^2(t-1)$$



Concatenation attention

- Attention – how much should each encoder state contribute?
- Instead of inner product between query ($\mathbf{a}^2(t-1)$) and keys (encoder states), use a general functional relationship.
- Concatenate query with the keys and pass the result through a multilayer subnetwork.
- Vertically stack, or concatenate, the query $\mathbf{a}^2(t-1)$ with the previous encoder states $\mathbf{A}^1(T)$.

$$\begin{aligned}\bar{\mathbf{A}}^{1,2}(t) &= \begin{bmatrix} \mathbf{A}^1(T) \\ [\mathbf{a}^2(t-1) \quad \mathbf{a}^2(t-1) \quad \dots \quad \mathbf{a}^2(t-1)] \end{bmatrix} \\ &= \begin{bmatrix} \begin{bmatrix} \mathbf{a}^1(T) \\ \mathbf{a}^2(t-1) \end{bmatrix} & \begin{bmatrix} \mathbf{a}^1(T-1) \\ \mathbf{a}^2(t-1) \end{bmatrix} & \dots & \begin{bmatrix} \mathbf{a}^1(T-D+1) \\ \mathbf{a}^2(t-1) \end{bmatrix} \end{bmatrix}\end{aligned}$$



Concatenation attention implementation

- The matrix $\bar{\mathbf{A}}^{1,2}(t)$ is passed into a two layer network.

$$\mathbf{A}^4(t) = \mathbf{tansig}(\mathbf{LW}^{4,2}\bar{\mathbf{A}}^{1,2}(t) + \mathbf{b}^4)$$

$$\mathbf{N}^5(t) = \mathbf{LW}^{5,4}\mathbf{A}^4(t) + \mathbf{b}^5$$

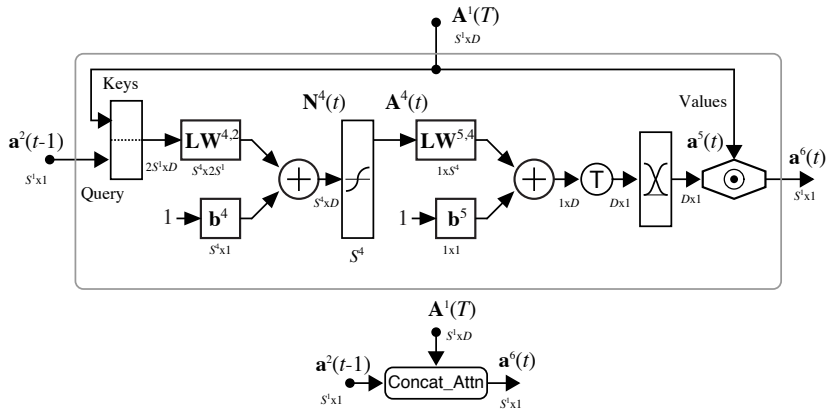
$$\mathbf{a}^5(t) = \mathbf{softmax}\left([\mathbf{N}^5(t)]^T\right)$$

- To get the attention weights, multiply by the values $\mathbf{A}^1(T)$.

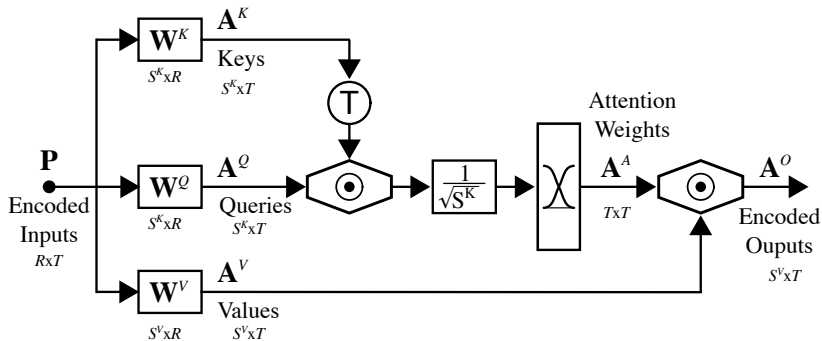
$$\mathbf{a}^6(t) = \mathbf{A}^1(T)\mathbf{a}^5(t)$$



Concatenation attention diagram



Self dot product attention (basis for the transformer)



Self attention example (1)

$$\mathbf{P} = [\mathbf{p}(1) \quad \mathbf{p}(2)] = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix}$$

$$\mathbf{W}^Q = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \mathbf{W}^K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \mathbf{W}^V = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{A}^Q = \mathbf{W}^Q \mathbf{P} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix} = [\mathbf{a}^Q(1) \quad \mathbf{a}^Q(2)]$$

$$\mathbf{A}^K = \mathbf{W}^K \mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix} = [\mathbf{a}^K(1) \quad \mathbf{a}^K(2)]$$

$$\mathbf{A}^V = \mathbf{W}^V \mathbf{P} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 2 & 0 \end{bmatrix} = [\mathbf{a}^V(1) \quad \mathbf{a}^V(2)]$$



Self attention example (2)

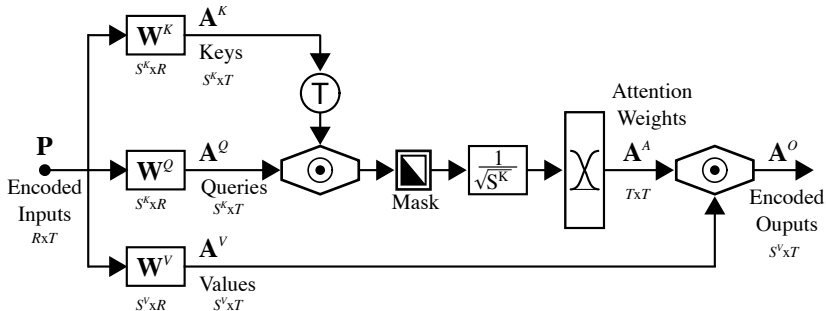
$$\begin{aligned}\mathbf{N}^A &= \frac{1}{\sqrt{2}} [\mathbf{A}^K]^T \mathbf{A}^Q = \frac{1}{\sqrt{2}} \begin{bmatrix} 0 & 2 \\ 2 & 0 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 2 & 0 \\ 2 & 4 \end{bmatrix} \\ &= \begin{bmatrix} \sqrt{2} & 0 \\ \sqrt{2} & 2\sqrt{2} \end{bmatrix} \\ &= \frac{1}{\sqrt{2}} \begin{bmatrix} \mathbf{a}^K(1)^T \\ \mathbf{a}^K(2)^T \end{bmatrix} [\mathbf{a}^Q(1) \quad \mathbf{a}^Q(2)] \\ &= \frac{1}{\sqrt{2}} \begin{bmatrix} \mathbf{a}^K(1)^T \mathbf{a}^Q(1) & \mathbf{a}^K(1)^T \mathbf{a}^Q(2) \\ \mathbf{a}^K(2)^T \mathbf{a}^Q(1) & \mathbf{a}^K(2)^T \mathbf{a}^Q(2) \end{bmatrix}\end{aligned}$$

$$\mathbf{A}^A = \text{softmax}(\mathbf{N}^A, \text{columns}) = \begin{bmatrix} 0.5 & 0.06 \\ 0.5 & 0.94 \end{bmatrix}$$

$$\begin{aligned}\mathbf{A}^O &= \mathbf{A}^V \mathbf{A}^A = [0.5\mathbf{a}^V(1) + 0.5\mathbf{a}^V(2) \quad 0.06\mathbf{a}^V(1) + 0.94\mathbf{a}^V(2)] \\ &= \begin{bmatrix} 1.5 & 1.94 \\ 1 & 0.12 \end{bmatrix}\end{aligned}$$



Masked attention

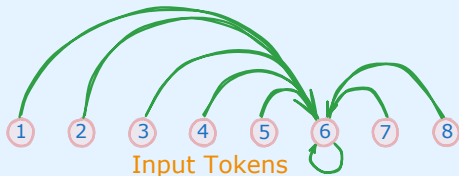


- The attention is masked so that tokens only attend to previous tokens.
- During text generation, we can't see future tokens.
- This is how GPT models generate text sequentially.

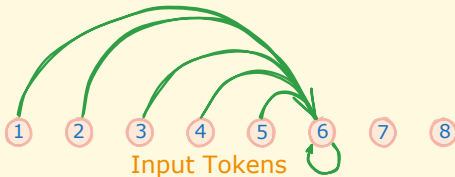


Full self-attention vs masked self-attention

Self-Attention



Masked Self-Attention



Masked attention example (1)

$$\mathbf{P} = [\mathbf{p}(1) \quad \mathbf{p}(2) \quad \mathbf{p}(3)] = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$\mathbf{W}^Q = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \mathbf{W}^K = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}, \mathbf{W}^V = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$$

$$\mathbf{A}^Q = \mathbf{W}^Q \mathbf{P} = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 1 & 0 & 0 \end{bmatrix} = [\mathbf{a}^Q(1) \quad \mathbf{a}^Q(2) \quad \mathbf{a}^Q(3)]$$

$$\mathbf{A}^K = \mathbf{W}^K \mathbf{P} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 2 & 1 \\ 2 & 0 & 1 \end{bmatrix} = [\mathbf{a}^K(1) \quad \mathbf{a}^K(2) \quad \mathbf{a}^K(3)]$$

$$\mathbf{A}^V = \mathbf{W}^V \mathbf{P} = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 0 & 1 \end{bmatrix} = [\mathbf{a}^V(1) \quad \mathbf{a}^V(2) \quad \mathbf{a}^V(3)]$$



Masked attention example (2)

$$[\mathbf{A}^K]^T \mathbf{A}^Q = \begin{bmatrix} 0 & 2 \\ 2 & 0 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 0 \\ 2 & 4 & 4 \\ 2 & 2 & 2 \end{bmatrix}$$

$$\mathbf{M} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{No token attends to future tokens}$$

$$\mathbf{M} \boxtimes [\mathbf{A}^K]^T \mathbf{A}^Q = \begin{bmatrix} 2 & 0 & 0 \\ -\infty & 4 & 4 \\ -\infty & -\infty & 2 \end{bmatrix}$$

$$\mathbf{N}^A = \frac{1}{\sqrt{2}} \mathbf{M} \boxtimes [\mathbf{A}^K]^T \mathbf{A}^Q = \begin{bmatrix} \sqrt{2} & 0 & 0 \\ -\infty & 2\sqrt{2} & 2\sqrt{2} \\ -\infty & -\infty & \sqrt{2} \end{bmatrix}$$

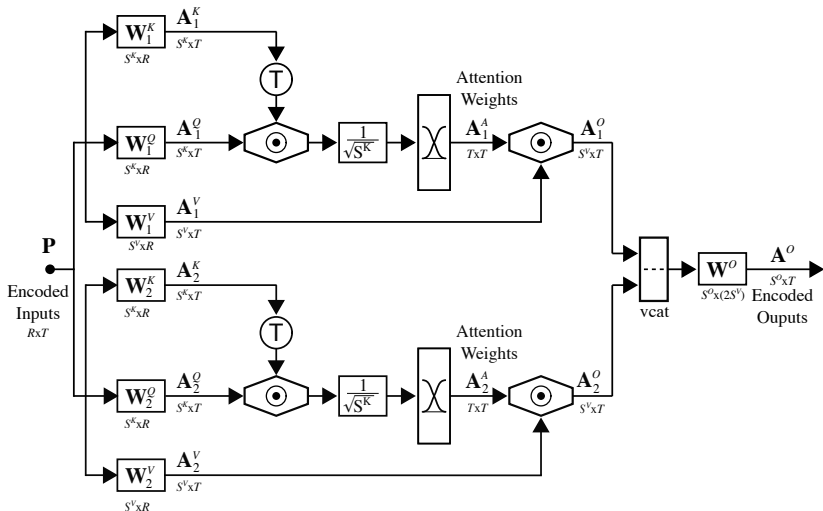
$$\mathbf{A}^A = \text{softmax}(\mathbf{N}^A, \text{columns}) = \begin{bmatrix} 1 & 0.06 & 0.04 \\ 0 & 0.94 & 0.77 \\ 0 & 0 & 0.19 \end{bmatrix}$$

$$\begin{aligned} \mathbf{A}^O &= \mathbf{A}^V \mathbf{A}^A = [\mathbf{a}^V(1) \quad 0.06\mathbf{a}^V(1) + 0.94\mathbf{a}^V(2) \quad 0.04\mathbf{a}^V(1) + 0.77\mathbf{a}^V(2) + 0.19\mathbf{a}^V(3)] \\ &= \begin{bmatrix} 1 & 1.94 & 1.77 \\ 2 & 0.11 & 0.28 \end{bmatrix} \end{aligned}$$

First column attends to first token.
Second column attends to first two tokens.



Multi-head attention (two head example)



- Often, $S^k = R/h$ and $S^O = R$, where $h = \#$ heads.



Inductive Bias of Attention

- Permutation invariance: The results of the dot product operation do not depend on where an input appears in the sequence – "The dog bit the cat." vs "The cat bit the dog."
- Locality agnosticism: Self-attention can directly relate any two elements in the sequence, regardless of their distance.
- Dot product similarity: The dot product measures the cosine similarity between query and key vectors, which biases the model to attend more to elements whose representations are similar in direction.
- Some of these cause issues that must be addressed – **input embedding** and **positional encoding**.



Why do we need input embedding?

- In the attention operation, dot products between input vectors determine how tokens interact.
- If the input is text, we need to convert words to numerical vectors.
- The dot product between these vectors should reflect relationships between words.
- Two key problems to solve:
 - How do we represent words as numbers?
 - How do we make the representation capture word relationships?



One-hot representation

- Suppose we have a vocabulary of $V = 5$ words:

Index	Word
1	the
2	cat
3	sat
4	on
5	mat

- A **one-hot** vector has a single element equal to 1 and the rest equal to 0:

$$\mathbf{o}(1) = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \text{ ("the")}, \quad \mathbf{o}(2) = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \text{ ("cat")}, \quad \mathbf{o}(3) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \text{ ("sat")}$$



Problems with one-hot vectors

- One-hot vectors have dimension V (the vocabulary size), which is typically tens of thousands.
- The one-hot representation does not capture any relationships between words.
- Every word is equally distant from every other word:

$$\mathbf{o}(j)^T \mathbf{o}(k) = \begin{cases} 1, & j = k \\ 0, & j \neq k \end{cases}$$

- We need a more compact, meaningful representation.



- The **embedding matrix** $\mathbf{W}^E$ (dimension $R \times V$) maps each one-hot vector to a dense vector:

$$\mathbf{p}(t) = \mathbf{W}^E \mathbf{o}(t)$$

- R is the chosen embedding dimension ($R \ll V$).
- Multiplying $\mathbf{W}^E$ by a one-hot vector simply **selects the corresponding column** of $\mathbf{W}^E$.
- In practice, implemented as a table lookup, not a matrix multiply.
- $\mathbf{W}^E$ is trained with all other network weights.



Embedding example (1)

Let $R = 3$, $V = 5$, and

$$\mathbf{W}^E = \begin{bmatrix} 0.2 & 0.8 & -0.1 & 0.5 & 0.3 \\ -0.5 & 0.3 & 0.7 & -0.2 & 0.6 \\ 0.1 & -0.4 & 0.2 & 0.9 & -0.3 \end{bmatrix}$$

Compute the embedding for each word in “the cat sat”:

$$\mathbf{p}(1) = \mathbf{W}^E \mathbf{o}(1) = \begin{bmatrix} 0.2 \\ -0.5 \\ 0.1 \end{bmatrix} \quad (\text{“the”} = \text{column 1})$$

$$\mathbf{p}(2) = \mathbf{W}^E \mathbf{o}(2) = \begin{bmatrix} 0.8 \\ 0.3 \\ -0.4 \end{bmatrix} \quad (\text{“cat”} = \text{column 2})$$

$$\mathbf{p}(3) = \mathbf{W}^E \mathbf{o}(3) = \begin{bmatrix} -0.1 \\ 0.7 \\ 0.2 \end{bmatrix} \quad (\text{“sat”} = \text{column 3})$$



Embedding example (2)

The resulting input matrix for “the cat sat” ($T = 3$):

$$\mathbf{P} = [\mathbf{p}(1) \quad \mathbf{p}(2) \quad \mathbf{p}(3)] = \begin{bmatrix} 0.2 & 0.8 & -0.1 \\ -0.5 & 0.3 & 0.7 \\ 0.1 & -0.4 & 0.2 \end{bmatrix}$$

- This $R \times T$ matrix is the input to the self-attention operation.
- During training, the columns of $\mathbf{W}^E$ are adjusted so that words with similar meanings have similar embedding vectors.
- For example, “cat” and “dog” would be closer together than “cat” and “sat.”
- This is related to the dot product similarity used in attention.



Why do we need positional encoding?

- Self-attention is **permutation invariant**: dot products between tokens depend only on their embedding vectors, not on where they appear in the sequence.
- If we swap “cat” and “sat” in the input, the embeddings are swapped, but the set of dot products remains the same.
- “The cat sat” and “the sat cat” would produce identical attention outputs (with corresponding outputs also swapped).
- Word order matters! The network needs a way to distinguish between different positions.



Adding positional information

- Add a position-dependent vector to each embedding:

$$\tilde{\mathbf{p}}(t) = \mathbf{p}(t) + \mathbf{p}^e(t)$$

- $\mathbf{p}(t)$ is the embedding vector.
- $\mathbf{p}^e(t)$ is the **positional encoding** vector for position t .
- $\tilde{\mathbf{p}}(t)$ replaces $\mathbf{p}(t)$ in the self-attention operation.
- Each position has a unique encoding vector, so the attention mechanism can distinguish between tokens at different positions.



Sinusoidal positional encoding

- The encoding should provide a unique signal for each position and allow the network to learn relative positions.
- Sinusoidal encoding (Vaswani et al., 2017) uses sine and cosine functions at different frequencies:

$$p_{2i}^e(t) = \sin\left(\frac{t}{10000^{2i/R}}\right)$$
$$p_{2i+1}^e(t) = \cos\left(\frac{t}{10000^{2i/R}}\right)$$

- $i = 0, 1, \dots$ indexes the dimension pairs.
- Even-indexed elements use sine, odd-indexed use cosine.
- Frequencies decrease geometrically from 1 (for $i = 0$) to $1/10000$ (for $i = R/2 - 1$).



Why sinusoidal encoding works

- Any fixed offset Δt can be represented as a linear transformation:

$$\mathbf{p}^e(t + \Delta t) = \mathbf{M}_{\Delta t} \mathbf{p}^e(t)$$

where $\mathbf{M}_{\Delta t}$ is a block-diagonal matrix of 2×2 rotation matrices.

- This makes it easy for the network to learn to attend to **relative positions** (e.g., “the word two positions to the left”).
- The multiple frequencies provide a rich position signal at many scales:
 - High-frequency pairs: distinguish nearby positions.
 - Low-frequency pairs: distinguish distant positions.



Positional encoding example (1)

Using $R = 3$, we have one sine/cosine pair ($i = 0$) and one sine ($i = 1$):

$$p_0^e(t) = \sin(t), \quad p_1^e(t) = \cos(t), \quad p_2^e(t) = \sin\left(\frac{t}{464.2}\right)$$

Computing for $t = 1, 2, 3$:

$$\mathbf{p}^e(1) = \begin{bmatrix} \sin(1) \\ \cos(1) \\ \sin(0.0022) \end{bmatrix} = \begin{bmatrix} 0.841 \\ 0.540 \\ 0.002 \end{bmatrix}, \quad \mathbf{p}^e(2) = \begin{bmatrix} 0.909 \\ -0.416 \\ 0.004 \end{bmatrix}$$
$$\mathbf{p}^e(3) = \begin{bmatrix} 0.141 \\ -0.990 \\ 0.006 \end{bmatrix}$$

First two elements (high frequency) vary rapidly; third element (low frequency) changes very slowly.



Positional encoding example (2)

Add positional encodings to the embeddings from the previous example:

$$\tilde{\mathbf{p}}(1) = \begin{bmatrix} 0.2 \\ -0.5 \\ 0.1 \end{bmatrix} + \begin{bmatrix} 0.841 \\ 0.540 \\ 0.002 \end{bmatrix} = \begin{bmatrix} 1.041 \\ 0.040 \\ 0.102 \end{bmatrix} \quad (\text{"the", pos 1})$$

$$\tilde{\mathbf{p}}(2) = \begin{bmatrix} 0.8 \\ 0.3 \\ -0.4 \end{bmatrix} + \begin{bmatrix} 0.909 \\ -0.416 \\ 0.004 \end{bmatrix} = \begin{bmatrix} 1.709 \\ -0.116 \\ -0.396 \end{bmatrix} \quad (\text{"cat", pos 2})$$

$$\tilde{\mathbf{p}}(3) = \begin{bmatrix} -0.1 \\ 0.7 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0.141 \\ -0.990 \\ 0.006 \end{bmatrix} = \begin{bmatrix} 0.041 \\ -0.290 \\ 0.206 \end{bmatrix} \quad (\text{"sat", pos 3})$$

The modified input matrix $\tilde{\mathbf{P}}$ replaces $\mathbf{P}$ in the self-attention equations.



Positional encoding breaks permutation invariance

- With “cat” at position 2: $\tilde{\mathbf{p}}(1)^T \tilde{\mathbf{p}}(2) = 1.735$
- If we swap “cat” to position 3:

$$\mathbf{p}(\text{“cat”}) + \mathbf{p}^e(3) = \begin{bmatrix} 0.8 \\ 0.3 \\ -0.4 \end{bmatrix} + \begin{bmatrix} 0.141 \\ -0.990 \\ 0.006 \end{bmatrix} = \begin{bmatrix} 0.941 \\ -0.690 \\ -0.394 \end{bmatrix}$$

- Now: $\tilde{\mathbf{p}}(1)^T \tilde{\mathbf{p}}(2) = 0.912$
- The dot product changed from 1.735 to 0.912.
- The attention weights will be different $\Rightarrow$ the network can learn to use word order.

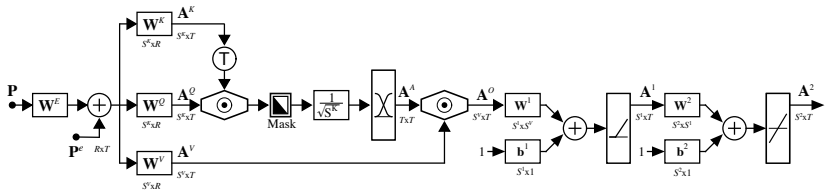


Alternatives to sinusoidal encoding

- **Learned positional embeddings:** Make $\mathbf{p}^e(t)$ trainable parameters, similar to the embedding matrix.
 - Used successfully in many models (e.g., BERT, GPT-2).
 - Cannot generalize to sequence lengths not seen during training.
- **Rotary Position Embedding (RoPE):** Encodes position by rotating the query and key vectors in the attention mechanism.
 - Encodes relative position directly into the dot product.
 - Used in many recent models (e.g., LLaMA).
- Sinusoidal encoding has the advantage of potentially generalizing to unseen sequence lengths.



Decoder-only transformer



Transformer generation

Input sequence

Output sequence

